1 实验一：熟悉 Oberon-0 语言定义

实验目的

本实验（实验一）旨在深入理解 Oberon-0 语言的定义，为后续词法分析、语法分析和语法制导翻译实验奠定基础。具体目标包括：

- 阅读 Oberon-0 语言的 BNF 定义和语义描述，掌握该语言的完整语法规则。
- 编写一个词法、语法和语义均正确的 Oberon-0 源程序，覆盖所有语法构造。
- 基于变异测试思想，针对每一种可能的词法、语法和语义错误构造变异程序。
- 讨论 Oberon-0 语言的特点（保留字与关键字的区别、表达式语法的独特性）。
- 分析 Oberon-0 文法的二义性，理解其与其他高级语言在语法设计上的差异。

实验目录与项目组织

实验一的项目文件均存放于 `ex1/` 子目录下，各文件说明如下：

- `readme.txt`：自述文件，包含姓名、学号、完成日期及补充说明。
- `testcases/BankSystem.obr`：词法、语法、语义均正确的 Oberon-0 源程序，覆盖了语言的全部语法构造。
- `testcases/BankSystem.001--022`：22 个变异程序（14 个直接构造 + 8 个 derived 派生），分别包含一种词法、语法或语义错误，首行注释标明期望异常类型和变异来源。文件扩展名以数字序号区分。

1.1 正确的 Oberon-0 源程序

根据 Oberon-0 语言的 BNF 定义，编写了一个模拟银行账户管理系统的 Oberon-0 源程序 `BankSystem.obr`，覆盖了 Oberon-0 语言的所有主要语法构造：

- **模块声明**：MODULE BankSystem ... END BankSystem.
- **常量声明**：CONST MaxAccounts = 50; MinBalance = 100;
- **类型声明**：TYPE AccountRec = RECORD ... END; TYPE AccountArray = ARRAY 50 OF AccountRec;
- **变量声明**：VAR accounts, count, i, choice, amount, accId: INTEGER; VAR running, found: BOOLEAN;
- **过程声明**（5 个）：InitAccounts、FindAccount（含 VAR 参数）、Deposit、

Withdraw、CreateAccount

- **IF 语句**：含 IF-THEN-ELSE、IF-THEN-ELSIF-THEN-ELSE-END 等完整形式
- **WHILE 循环语句**
- **赋值语句**:含数组下标选择(`accounts[k].id := 0`)和记录域选择(`accounts[k].balance := 0`)
- **过程调用**：含实际参数传递
- **预定义过程调用**：Read、Write、WriteLn
- **布尔变量与逻辑运算**： `found`、`(j < count) & found`

程序具有实际意义：实现银行账户的创建、存款、取款等基本操作，展示了 Oberon-0 作为教学语言的核心抽象能力。

1.2 变异程序

变异程序	类别	错误描述	检测结果
BankSystem.001	词法	非法符号 $	IllegalSymbolException ✓
BankSystem.002	词法	标识符超 24 字符	IllegalIdentifierLengthException ✓
BankSystem.003	词法	非法八进制 08	IllegalOctalException ✓
BankSystem.004	词法	25id 无空白分隔	IllegalIntegerException ✓
BankSystem.005	词法	注释未闭合	MismatchedCommentException ✓
BankSystem.014	词法	整数超 12 位	IllegalIntegerRangeException ✓
BankSystem.006	语法	缺左括号	MissingLeftParenthesisException ✓
BankSystem.007	语法	缺右括号	MissingRightParenthesisException ✓
BankSystem.008	语法	两表达式间缺运算符	MissingOperatorException ✓
BankSystem.009	语法	二元运算缺右操作数	MissingOperandException ✓
BankSystem.010	语法	缺 END	SyntacticException ✓
BankSystem.011	语义	INTEGER & BOOLEAN	TypeMismatchedException ✓
BankSystem.012	语义	参数数目不匹配	ParameterMismatchedException ✓
BankSystem.013	语义	模块名不匹配	SemanticException ✓
BankSystem.015	词法	非法符号 @	IllegalSymbolException ✓
BankSystem.022	词法	非法八进制 09	IllegalOctalException ✓
BankSystem.021	词法	注释未闭合	MismatchedCommentException ✓
BankSystem.017	语法	缺左括号	MissingLeftParenthesisException ✓
BankSystem.016	语法	缺右括号	MissingRightParenthesisException ✓
BankSystem.020	语法	缺 END	SyntacticException ✓
BankSystem.018	语义	& 用于 INTEGER	TypeMismatchedException ✓
BankSystem.019	语义	参数数目不匹配	ParameterMismatchedException ✓

表 1.1 变异程序及其错误检测结果

基于变异测试思想，编写了 22 个变异程序（其中 8 个为从 BankSystem.obr 直接最小改动派生的 derived 系列），覆盖 Oberon-0 语言全部 6 种词法错误、4 种

语法错误和 2 种语义错误。每个变异程序首行以注释标注期望的错误类型和变异来源。

1.3 Oberon-0 语言特点讨论

1.3.1 保留字与关键字的区别

Oberon-0 语言区别了保留字（Reserved Word）与关键字（Keyword）：

- **保留字**（如 MODULE、END、PROCEDURE、IF、THEN、WHILE、DO、DIV、MOD、OR 等）是语法结构的骨架部分，**不允许**被程序员用作标识符。在词法分析阶段即被识别为独立 Token 类型。
- **关键字**（如 INTEGER、BOOLEAN、Read、Write、WriteLn、TRUE、FALSE 等）具有预定义语义但**允许**在嵌套作用域中被重新声明而改变含义。在词法分析阶段被识别为标识符（TK_IDENT），其特殊语义在语法/语义分析阶段处理。本实验的扫描器将 Read、Write、WriteLn 识别为标识符 Token，由语法分析器根据上下文确定其语义角色。

1.3.2 表达式语法与 Java/C++ 的差异

Oberon-0 表达式语法与 Java/C++ 的主要差异：

1) **层次化编码优先级**：Oberon-0 通过 expression → simple_expression → term → factor 四层产生式自然编码了运算符优先级，无需额外的优先级表。而 C/Java 有 15+ 级优先级，完全依赖优先级约定而非文法结构。
2) **逻辑运算符符号不同**：Oberon-0 使用 &（与）、OR（或）、~（非），不同于 C/Java 的 &&、||、!。
3) **无短路求值语义**：Oberon-0 未明确规定短路求值，而 C/Java 的 && 和 || 具有短路语义。
4) **运算符集合少**：不支持位运算、自增自减、三目运算符、复合赋值。

1.4 Oberon-0 文法二义性讨论

经分析，Oberon-0 的 BNF 定义**不存在二义性**：

1) **悬挂 else 问题已被消除**：Oberon-0 的 IF 语句使用 IF-THEN-ELSIF-ELSE-**END** 闭合结构，每个 IF 必须有显式的 END 结束标记。这从根本上避免了 Pascal/C 等语言中 else 与哪个 if 匹配的经典二义性。Wirth 在其语言设计

中一贯坚持这一理念。

2) **运算符优先级内置于文法层次**：四层表达式文法结构（expression → simple_expression → term → factor）已明确编码了关系运算、加减运算、乘除运算的优先级关系，无需额外的优先级和结合性规则。
3) **运算符重载通过位置消歧**：一元 - 出现在 simple_expression 起始的 ["+" | "-"] 中，二元 - 出现在（"+" | "-" | "OR"）中，语法位置自然区分。

1.5 自我评分参照

参照实验一评价标准，各项完成情况如下：

项目	权重 (%)	对应内容
1、正确的 Oberon-0 源程序	20	3.1 节 BankSystem.obr
2、变异程序	40	3.2 节 22 个变异程序（含 8 个 derived）
3、语言特点讨论	10	3.3 节保留字/关键字、表达式差异
4、文法二义性讨论	20	3.4 节无二义性证明
5、文档组织及 readme	10	ex1/readme.txt

表 1.2 实验一自我评分参照

1.6 实验心得体会

通过实验一，我对 Oberon-0 语言有了系统而深入的理解。编写覆盖全部语法构造的正确源程序 BankSystem.obr 的过程，让我逐一梳理了 Oberon-0 的模块结构、声明体系、控制流构造和表达式语法。在设计变异程序时，我深刻体会到变异测试" 最小改动、精确触发" 的思想——每个变异仅改变程序的一处细节，便能准确揭露特定类别的错误检测需求。

Oberon-0 语法设计的简洁与严谨给我留下了深刻印象。Wirth 通过 IF-END 显式闭合结构从根本上消除了悬挂 else 问题，通过四层表达式文法将运算符优先级自然编码在语法中——这些设计使得 Oberon-0 成为一门分析友好、教学价值极高的语言。与 Java/C++ 等工业语言的对比也让我更清楚地看到，语言的" 简洁性" 与" 表达能力" 之间始终存在权衡。